

Portfolio Pro

Sistema de Evaluación por Competencias

SOPORTE MATERIAL DE LA OBRA

Fragmentos representativos del código fuente original

Para fines de registro ante la Oficina Nacional de Derecho de Autor (ONDA)

República Dominicana

Nombre de la obra:	Portfolio Pro (Noether SaaS)
Tipo:	Programa de Computadora (Software como Servicio — SaaS)
Tecnologías:	TypeScript, React, Node.js, PostgreSQL (Supabase)
Fecha de generación:	30 de julio de 2026

Aviso de seguridad: Toda credencial, llave API, token de base de datos, contraseña u otro dato sensible ha sido reemplazado por *****. Este documento contiene únicamente lógica de negocio y código fuente original del autor. Se han excluido librerías externas (node_modules), archivos de configuración genéricos y dependencias de terceros.

Índice de Contenidos

1. **Sección 1 — Modelo de Datos:** Definiciones de tipos e interfaces del dominio
2. **Sección 2 — Estado Global (Store):** Gestor centralizado del estado de la aplicación
3. **Sección 3 — Capa de Datos:** Conexión a base de datos y sincronización en tiempo real
4. **Sección 4 — Lógica de Negocio:** Evaluaciones, calificaciones, recuperaciones y cursos
5. **Sección 5 — Enrutamiento:** Definición de rutas de la aplicación
6. **Sección 6 — Punto de Entrada:** Orquestador principal de la aplicación
7. **Sección 7 — Pantallas Principales:** Curso Detalle y Autenticación
8. **Sección 8 — Componentes Especializados:** Rúbrica de evaluación y Acta de recuperación
9. **Sección 9 — Constantes del Dominio:** Competencias, asignaturas y escala académica
10. **Sección 10 — Plantillas de Boletines:** Generación de informes escolares oficiales
11. **Sección 11 — Backend (Servidor):** API de autenticación y recuperación de contraseñas
12. **Sección 12 — Esquema de Base de Datos:** Políticas de seguridad y estructura SQL

Sección 1 — Modelo de Datos

src/types/index.ts

Define todas las interfaces TypeScript del dominio educativo: estudiantes, cursos, actividades, calificaciones, competencias, boletines, evaluaciones de rúbrica/cotejo, incidencias, perfiles docentes y el estado global de la aplicación.

```
export type Screen =
  | 'inicio' | 'dashboard' | 'cursos' | 'curso-detalle'
  | 'incidencias' | 'planificacion' | 'comunidad'
  | 'rubrica' | 'cotejo' | 'estudiante' | 'calificaciones-anuales' | 'ajustes';

export type Nivel = 1 | 2 | 3 | 4;

export const COMPETENCIAS_LABEL = {
  BC1: "Comunicativa",
  BC2: "Pensamiento lógico, creativo y crítico; resolución de problemas",
  BC3: "Científica y tecnológica; ambiental y de la salud",
  BC4: "Desarrollo personal y espiritual; ética y ciudadana",
} as const;

export type Competencia = keyof typeof COMPETENCIAS_LABEL;

export interface BCScore { nivel: Nivel; puntaje: number; }

export interface Estudiante {
  id: number; nombre: string; apellido: string; avatarColor: string;
  cursoId: number; groupId: number; sharedCourseId?: string;
  nivel: Nivel; puntaje: number;
  bc1: BCScore; bc2: BCScore; bc3: BCScore; bc4: BCScore;
  actividadesRecientes: number; enRiesgo: boolean;
  userId?: string; numeroLista: number;
}

export interface Curso {
  id: number; nombre: string; asignatura: string;
  grado: string; seccion: string; periodo: string;
  diasSemana: string[]; color: string;
  isTutorOficial?: boolean; userId?: string;
  groupId: number; sharedCourseId?: string;
  configuracionEvaluacion?: Record<string, unknown>;
}

export interface Actividad {
  id: number; nombre: string; cursoId: number;
  fecha: string; periodo: string;
  bcAsignados: Competencia[];
  secuenciaId?: number; isRec?: boolean;
}

export interface CalificacionActividad {
  id?: number; cursoId: number; estudianteId: number;
  actividadId: number; userId: string; asignatura: string;
  periodo: string; competencias: Competencia[];
  descriptores: string[]; puntaje: number | null;
  recuperacion: number | null; sharedCourseId?: string;
}

export interface RecuperacionBC {
  estudianteId: number; cursoId: number;
  bc: 1 | 2 | 3 | 4; puntaje: number | null;
  periodo: string; sharedCourseId?: string;
  asignatura?: string; userId?: string;
}

export interface EvaluacionRubrica {
  id: number; estudianteId: number; actividadId: number;
  cursoId: number; fecha: string;
  selecciones: Partial<Record<Competencia, Nivel>>;
}
```

```

    observaciones?: string; puntajeTotal: number;
    plantillaId?: number | null;
}

export interface AppState {
  cursos: Curso[]; estudiantes: Estudiante[];
  incidencias: Incidencia[]; actividades: Actividad[];
  calificaciones: CalificacionActividad[];
  recuperaciones: RecuperacionBC[];
  secuencias: Secuencia[]; eventos: EventoCalendario[];
  posts: Post[]; descriptoresRubrica: DescriptorRubrica[];
  nivelesPuntaje: NivelPuntaje[];
  evaluacionesRubrica: EvaluacionRubrica[];
  evaluacionesCotejo: EvaluacionCotejo[];
  docentes: Docente[]; plantillas: Plantilla[];
  cursoDetalle: CursoDetalleEvaluacion[];
  perfiles: UserProfile[]; notificaciones: Notification[];
  cursoDocentes: CursoDocente[]; grupos: Grupo[];
}

```

Sección 2 — Estado Global (Store)

src/store/appStore.ts

Almacén centralizado Zustand que gestiona el estado completo de la aplicación: datos académicos, sesión de usuario, configuración de interfaz y rúbricas flotantes. Solo persiste preferencias de UI, no datos sensibles.

```

import { create } from 'zustand';
import { persist } from 'zustand/middleware';
import type { Session } from '@supabase/supabase-js';
import type { AppState, SearchResults, FloatingRubricWindow, Nivel } from '../types';
import { initialState } from '../data/mockData';

interface AppStore {
  state: AppState;
  session: Session | null;
  loading: boolean;
  darkMode: boolean;
  selectedCursoId: number | null;
  selectedEstudianteId: number | null;
  genericToast: { message: string; type: 'success' | 'error' } | null;
  floatingRubrics: FloatingRubricWindow[];
  activeRubricSelection: Record<string, Nivel>;
  // Actions
  setAppState: (state: AppState | ((prev: AppState) => AppState)) => void;
  setSession: (session: Session | null) => void;
  setLoading: (loading: boolean) => void;
}

export const useAppStore = create<AppStore>()(
  persist(
    (set) => ({
      state: initialState,
      session: null,
      loading: true,
      darkMode: false,
      // ... demás valores iniciales
      setAppState: (updater) => set((prev) => ({
        state: typeof updater === 'function' ? updater(prev.state) : updater
      })),
      setState: (updater) => set((prev) => ({
        state: typeof updater === 'function' ? updater(prev.state) : updater
      })),
    }),
    {
      name: 'terra-cognita-storage',
      partialize: (state) => ({
        selectedCursoId: state.selectedCursoId,
        selectedEstudianteId: state.selectedEstudianteId,

```

```

        darkMode: state.darkMode
      }},
    }
  )
};

```

Sección 3 — Capa de Datos y Sincronización en Tiempo Real

src/lib/supabase.ts

Inicialización del cliente de base de datos con wrapper de fetch personalizado para diagnóstico de errores de red y detección de bloqueos CORS.

```

import { createClient } from '@supabase/supabase-js';

const supabaseUrl = '*****';
const supabaseAnonKey = '*****';

// Wrapper de fetch personalizado para auditoría de red
const customFetch = async (url: RequestInfo | URL, options?: RequestInit): Promise<Response> => {
  try {
    const res = await fetch(url, options);
    return res;
  } catch (err: any) {
    console.error(`[Supabase API Client] Fallo: ${url}. ${err?.message}`);
    throw err;
  }
};

export const supabase = createClient(supabaseUrl, supabaseAnonKey, {
  global: { fetch: customFetch }
});

```

src/hooks/useSupabaseData.ts (fragmento principal)

Hook maestro que sincroniza 25 tablas de Supabase con el estado React. Incluye suscripciones en tiempo real (Realtime) y mapeo camelCase ↔ snake_case entre frontend y base de datos.

```

export function useSupabaseData() {
  const { state, setAppState: setState, loading, setLoading, session } = useAppStore();

  const fetchData = useCallback(async (isSilent = false) => {
    if (!session?.user?.id) return;
    if (!isSilent) setLoading(true);
    try {
      const results = await Promise.all([
        supabase.from('perfiles').select('*', centros!perfiles_centro_id_fkey(*)'),
        supabase.from('cursos').select('*'),
        supabase.from('estudiantes').select('*'),
        supabase.from('actividades').select('*'),
        supabase.from('calificaciones').select('*'),
        supabase.from('recuperaciones').select('*'),
        // ... 19 tablas adicionales
      ]);

      // Detección y reporte de errores por tabla
      results.forEach((result, idx) => {
        if (result.error) {
          console.error(`[fetchData] Error en tabla "${tableNames[idx]}":`, result.error);
        }
      });

      // Protección: si una consulta falla, conservar estado previo
      const calificaciones = results[4].error ? null : results[4].data;
      const recuperaciones = results[5].error ? null : results[5].data;

      setState(prev => ({

```

```

    ...prev,
    // Mapeo de cada tabla con transformación snake_case → camelCase
    calificaciones: calificaciones === null ? prev.calificaciones
      : calificaciones.map((c) => ({ /* ... mapeo ... */ })),
    recuperaciones: recuperaciones === null ? prev.recuperaciones
      : recuperaciones.map((r) => ({ /* ... mapeo ... */ })),
  }));
} finally { if (!isSilent) setLoading(false); }
}, [session]);

// Suscripciones Realtime a cambios en la base de datos
useEffect(() => {
  if (session) {
    fetchData();
    const channel = supabase.channel(`db-changes-${session.user.id}`)
      .on('postgres_changes', { event: '*', schema: 'public', table: 'calificaciones' }, () => fetchData(true))
      .on('postgres_changes', { event: '*', schema: 'public', table: 'recuperaciones' }, () => fetchData(true))
      .on('postgres_changes', { event: '*', schema: 'public', table: 'cursos' }, () => fetchData(true))
      .subscribe();
    return () => supabase.removeChannel(channel);
  }
}, [session, fetchData]);
}

```

Sección 4 — Lógica de Negocio

src/hooks/useEvaluationActions.ts (fragmento: guardado de calificaciones y recuperaciones)

Lógica central de negocio para persistir calificaciones y actas de recuperación de competencias. Implementa upsert con resolución de conflictos por clave compuesta (estudiante + actividad) y actualización optimista del estado.

```

const saveCalificaciones = useCallback(async (
  califs: CalificacionActividad[],
  recs: RecuperacionBC[],
  cursoIdOverride?: number | null,
) => {
  const cursoId = cursoIdOverride ?? selectedCursoId;
  if (cursoId === null || !session?.user?.id) return;

  const dbCalifs = califs.map(c => ({
    user_id: session.user.id,
    curso_id: cursoId,
    estudiante_id: c.estudianteId,
    actividad_id: c.actividadId,
    asignatura: c.asignatura,
    periodo: c.periodo,
    competencias: c.competencias,
    descriptores: c.descriptores || [],
    puntaje: c.puntaje,
    recuperacion: c.recuperacion,
    shared_course_id: c.sharedCourseId || String(cursoId)
  }));

  const dbRecs = recs.map(r => ({
    user_id: session.user.id,
    estudiante_id: r.estudianteId,
    curso_id: r.cursoId,
    bc: r.bc,
    puntaje: r.puntaje,
    periodo: r.periodo,
    shared_course_id: r.sharedCourseId || String(r.cursoId),
    asignatura: r.asignatura || ''
  }));

  const promises = [];
  if (dbCalifs.length > 0)
    promises.push(supabase.from('calificaciones').upsert(dbCalifs,
      { onConflict: 'estudiante_id,actividad_id' }));
  if (dbRecs.length > 0)

```

```

    promises.push(supabase.from('recuperaciones').upsert(dbRecs,
      { onConflict: 'estudiante_id,curso_id,bc,periodo,asignatura' }));
  }

  if (promises.length > 0) {
    await Promise.all(promises);
    // Actualización optimista del estado global
    setState(s => {
      const nextCalifs = [...s.calificaciones];
      califs.forEach(newC => {
        const idx = nextCalifs.findIndex(oc =>
          oc.estudianteId === newC.estudianteId && oc.actividadId === newC.actividadId);
        if (idx !== -1) nextCalifs[idx] = newC; else nextCalifs.push(newC);
      });
      const nextRecs = [...s.recuperaciones];
      recs.forEach(nr => {
        const idx = nextRecs.findIndex(or =>
          or.estudianteId === nr.estudianteId && or.bc === nr.bc
          && or.periodo === nr.periodo && or.asignatura === nr.asignatura);
        if (idx !== -1) nextRecs[idx] = nr; else nextRecs.push(nr);
      });
      return { ...s, calificaciones: nextCalifs, recuperaciones: nextRecs };
    });
  }
}, [session, selectedCursoId, state, setState]);

```

src/hooks/useCursoDetalleData.ts (fragmento: cálculo de competencias por BC)

Motor de cálculo de promedios por competencia fundamental (BC1–BC4) con lógica de recuperación integrada: si el promedio de una competencia es inferior a 70 y existe un acta de recuperación, el puntaje de recuperación reemplaza el promedio. Incluye auto-guardado con debounce de 1 segundo.

```

export function useCursoDetalleData({ state, cursoId, currentUserId, onSaveCalificaciones }: Params) {
  const [localCalifs, setLocalCalifs] = useState<CalificacionActividad[]>([]);
  const [localRecs, setLocalRecs] = useState<RecuperacionBC[]>([]);
  const [isDirty, setIsDirty] = useState(false);

  // Sincronización bidireccional: estado global → estado local
  useEffect(() => {
    setLocalCalifs(state.calificaciones.filter(c => /* filtro por curso */));
    setLocalRecs(state.recuperaciones.filter(r =>
      (r.cursoId === cursoId || r.sharedCourseId === curso?.sharedCourseId)
      && (r.userId === currentUserId || !r.userId)));
  }, [state.calificaciones, state.recuperaciones, cursoId]);

  // Auto-save con debounce de 1 segundo
  useEffect(() => {
    if (!isDirty || !onSaveCalificaciones) return;
    const timer = setTimeout(async () => {
      await onSaveCalificaciones(localCalifs, localRecs, cursoId);
      setIsDirty(false);
    }, 1000);
    return () => clearTimeout(timer);
  }, [localCalifs, localRecs, isDirty]);

  // Cálculo de promedios por competencia con lógica de recuperación
  const enhancedEstudiantes = useMemo(() => {
    return filteredStudents.map(est => {
      const bcs: BCKey[] = ['BC1', 'BC2', 'BC3', 'BC4'];
      const bcValues = bcs.map((bc, idx) => {
        const bcNum = (idx + 1) as 1 | 2 | 3 | 4;
        // Filtrar actividades que NO sean recuperación y que tengan esta BC
        const actsBc = actividades.filter(a =>
          a.nombre !== 'Recuperación' && bcSel[a.id]?.has(bc));
        const rawScores = actsBc.map(a =>
          localCalifs.find(c => c.estudianteId === est.id
            && c.actividadId === a.id)?.puntaje ?? 0);
        const avg = actsBc.length
          ? Math.round(rawScores.reduce((a, b) => a + b, 0) / actsBc.length)
          : null;
        // Buscar acta de recuperación para esta BC y período
        const rec = localRecs.find(r =>
          r.estudianteId === est.id && Number(r.bc) === bcNum
          && r.periodo === selectedPeriodo)?.puntaje ?? null;
      });
    });
  }, [filteredStudents, actividades, bcSel, selectedPeriodo]);

```

```

        // Regla: Si hay recuperación Y el promedio era < 70, usar recuperación
        const final = (rec !== null && (avg === null || avg < 70)) ? rec : avg;
        return { bc, avg, rec, final };
    });
    return { ...est, bcValues, promTotal };
  });
}, [state.estudiantes, actividades, localCalifs, localRecs, selectedPeriodo]);
}

```

src/hooks/useCourseActions.ts (fragmento: creación de curso con grupo académico)

Lógica de creación de cursos con resolución automática de grupo académico (grado + sección) y vinculación de co-docentes mediante notificaciones pendientes.

```

const handleAddCurso = useCallback(async (c: Omit<Curso, 'id' | 'grupoId'>) => {
  if (!session?.user?.id) return null;

  // Upsert de grupo: si el grado+sección ya existe, reutilizar el grupo
  const { data: grupoData } = await supabase
    .from('grupos')
    .upsert({
      nombre: `${c.grado} ${c.seccion}`,
      grado: c.grado,
      seccion: c.seccion
    }, { onConflict: 'grado,seccion' })
    .select();

  const grupo_id = grupoData[0].id;
  const shared_course_id = c.sharedCourseId || crypto.randomUUID();

  const { data } = await supabase.from('cursos').insert([
    {
      nombre: c.nombre, asignatura: c.asignatura,
      grado: c.grado, seccion: c.seccion, periodo: c.periodo,
      dias_semana: c.diasSemana, color: c.color,
      is_tutor_oficial: c.isTutorOficial,
      configuracion_evaluacion: c.configuracionEvaluacion || {},
      user_id: session.user.id, shared_course_id, grupo_id
    }
  ]).select();

  // Vinculación automática de co-docentes pendientes
  const pendingNotif = state.notificaciones.find(n =>
    n.tipo === 'solicitud_vinculacion' && n.estado === 'pendiente'
    && n.grado === mapped.grado && n.seccion === mapped.seccion);
  if (pendingNotif) {
    await supabase.from('curso_docentes').insert([
      {
        curso_id: tutorCurso.id,
        docente_id: session.user.id,
        rol: 'co-docente',
        asignatura: mapped.asignatura
      }
    ]);
  }
}, [session, state.cursos, setState]);

```

Sección 5 — Enrutamiento de la Aplicación

src/AppRoutes.tsx (fragmento: definición de rutas)

Definición centralizada de todas las rutas de la SPA con wrappers que sincronizan el cursold de la URL con el estado global. Incluye rutas para: Inicio, Dashboard, Cursos, Detalle de Curso, Rúbricas, Cotejo, Estudiante, Calificaciones Anuales, Planificación, Comunidad, Incidencias y Boletines.

```

const CourseDetailRouteWrapper: React.FC<{
  setSelectedCursoId: (id: number | null) => void;
  renderCourseDetail: () => React.ReactElement;
}> = ({ setSelectedCursoId, renderCourseDetail }) => {
  const { id } = useParams<{ id: string }>();
  React.useEffect(() => {

```



```

    if (id) { setSelectedCursoId(Number(id)); }
  }, [id, setSelectedCursoId]);
  return renderCourseDetail();
};

// Rutas principales del sistema
<Routes>
  <Route path="/" element={<Inicio />} />
  <Route path="/dashboard" element={<Dashboard />} />
  <Route path="/cursos" element={<Cursos />} />
  <Route path="/curso-detalle/:id?" element={<CursoDetalle />} />
  <Route path="/rubrica" element={<Rubrica />} />
  <Route path="/cotejo" element={<Cotejo />} />
  <Route path="/estudiante" element={<Estudiante />} />
  <Route path="/calificaciones-anuales/:id?" element={...} />
  <Route path="/planificacion" element={<Planificacion />} />
  <Route path="/comunidad" element={<Comunidad />} />
  <Route path="/incidencias" element={<Incidencias />} />
  <Route path="/print-boletines/:cursoId" element={<PrintBoletines />} />
  <Route path="/ajustes" element={<ProfileSettings />} />
  <Route path="/reset-password" element={<ResetPassword />} />
</Routes>

```

Sección 6 — Punto de Entrada de la Aplicación

src/App.tsx

Componente raíz que orquesta la inicialización de datos, autenticación, composición de acciones (hooks), determinación de roles docentes, filtrado de datos visibles por asignatura y renderización condicional del layout completo.

```

export default function App() {
  const { state, loading, session, selectedCursoId } = useAppStore(useShallow(s => ({
    state: s.state, loading: s.loading, session: s.session,
    selectedCursoId: s.selectedCursoId,
  })));

  // Composición de todas las acciones del sistema
  const actions = {
    ...useCourseActions(),    ...useStudentActions(),
    ...useProfileActions(),   ...useEvaluationActions(),
    ...useIncidenciaActions(), ...useSecuenciaActions(),
    ...usePostActions(),      ...useNotificationActions(),
    ...useSupabaseData()
  };

  // Determinación del rol del docente actual en el curso seleccionado
  const currentCourseRole = useMemo(() => {
    const linked = state.cursoDocentes.find(cd =>
      cd.cursoId === selectedCursoId && cd.userId === session.userId);
    if (linked) return linked;
    const curso = state.cursos.find(c => c.id === selectedCursoId);
    if (curso?.userId === session.userId)
      return { rol: 'tutor', asignatura: curso.asignatura };
    return null;
  }, [selectedCursoId, session, state.cursoDocentes, state.cursos]);

  if (!session) return <Auth onAuthSuccess={() => actions.refresh()} />;

  return (
    <Layout>
      <AppRoutes {...actions} state={state} session={session} />
      <FloatingRubricManager />
    </Layout>
  );
}

```

Sección 7 — Pantallas Principales

src/screens/CursoDetalle.tsx

Pantalla central del sistema donde el docente visualiza y edita calificaciones en una tabla interactiva estilo hoja de cálculo. Soporta modos de pintura por puntos, edición directa de celdas, asignación de competencias por actividad, eliminación masiva, y vinculación de co-docentes.

```
export default function CursoDetalle(props: Props) {
  const { id } = useParams();
  const cursoId = Number(id) || props.cursoId || 0;
  const currentUserId = props.currentUserId || store.session?.user?.id;

  const {
    curso, selectedPeriodo, actividades, enhancedEstudiantes,
    localCalifs, localRecs, setCalif, setRec,
    isPointMode, isDragging, isDirty, isSaving,
    bcSel, setBcSel, triggerAnimation
  } = useCursoDetalleData({ state, cursoId, currentUserId,
    onSaveCalificaciones: props.onSaveCalificaciones });

  return (
    <div className="fixed inset-0 z-50 flex flex-col">
      <CursoDetalleHeader
        curso={curso} isDirty={isDirty} isSaving={isSaving}
        onSave={onSave} isPointMode={isPointMode}
        selectedPeriodo={selectedPeriodo}
        onAddActividad={onAddActividad} />
      <GradeTable
        actividades={actividades}
        estudiantes={enhancedEstudiantes}
        bcSel={bcSel}
        onSetGrade={({ estId, actId, val }) => {
          setCalif(estId, actId, val);
          if (val !== null) triggerAnimation(estId, actId, val);
        }} />
      <RubricModal
        targetEst={targetEst}
        onSetRec={setRec}
        selectedPeriodo={selectedPeriodo} />
      <VincularDocentesModal
        state={state} cursoId={cursoId} />
    </div>
  );
}
```

Sección 8 — Componentes Especializados

src/components/curso-detalle/RubricModal.tsx

Componente de Acta de Recuperación de Competencias. Presenta una tabla donde cada fila es una competencia fundamental (BC1–BC4). Solo habilita recuperación para competencias con promedio inferior a 70. El docente selecciona un nivel (Estratégico/90, Autónomo/80, Resolutivo/70, Receptivo/BC actual) mediante radio buttons visuales con colores institucionales.

```
const RubricModal: React.FC<RubricModalProps> = ({
  targetEst, rubricTarget, onClose, onSetRec, selectedPeriodo
}) => {
  if (!rubricTarget || !targetEst) return null;
  return (
    <div className="fixed inset-0 z-100 flex items-center justify-center">
      <table>
        <thead>
          <tr>
            <th>Competencia Deficiente</th>
            <th>Estratégico (90)</th>
            <th>Autónomo (80)</th>
          </tr>
        </thead>
      </table>
    </div>
  );
}
```

```

        <th>Resolutivo (70)</th>
        <th>Receptivo (BC actual)</th>
      </tr>
    </thead>
    <tbody>
      {targetEst.bcValues.map((v, idx) => {
        const bcIdx = (idx + 1) as 1 | 2 | 3 | 4;
        const isPassed = v.avg !== null && v.avg >= 70;
        // Deshabilitar fila si la competencia ya está aprobada
        return (
          <tr className={isPassed ? 'opacity-40 pointer-events-none' : ''}>
            <td>{v.bc} — BC: {v.avg ?? '--'}</td>
            {[90, 80, 70, v.avg || 0].map((score) => (
              <td>
                <button
                  onClick={() => onSetRec(targetEst.id, bcIdx, score)}
                  className={v.rec === score ? 'selected' : ''}
                />
              </td>
            ))}
          </tr>
        );
      })}
    </tbody>
  </table>
</div>
);
};

```

Sección 9 — Constantes del Dominio Educativo

src/constants/asignaturas.ts

Catálogo oficial de asignaturas del sistema educativo dominicano con funciones de búsqueda bidireccional (id ↔ nombre).

```

export const ASIGNATURAS_CATALOGO: Asignatura[] = [
  { id: 'lengua_espanola', nombre: 'Lengua Española' },
  { id: 'ingles', nombre: 'Lenguas Extranjeras (Inglés)' },
  { id: 'frances', nombre: 'Lenguas Extranjeras (Francés)' },
  { id: 'matematica', nombre: 'Matemática' },
  { id: 'ciencias_sociales', nombre: 'Ciencias Sociales' },
  { id: 'ciencias_naturaleza', nombre: 'Ciencias de la Naturaleza' },
  { id: 'artistica', nombre: 'Educación Artística' },
  { id: 'educacion_fisica', nombre: 'Educación Física' },
  { id: 'formacion_integral', nombre: 'Formación Integral Humana y Religiosa' },
];

```

src/utls/academic.ts

Escala de clasificación académica por competencias con colores institucionales propios del sistema.

```

export const getGradeClass = (score: number | null) => {
  if (score === null) return 'bg-slate-50 text-slate-400';
  if (score >= 90) return 'bg-[#7C9672]/10 text-[#7C9672]'; // Estratégico
  if (score >= 80) return 'bg-[#D8B55A]/15 text-[#8C6D1F]'; // Autónomo
  if (score >= 70) return 'bg-[#CB4834]/10 text-[#CB4834]'; // Resolutivo
  return 'bg-[#3F3C36]/10 text-[#3F3C36]'; // Receptivo
};

```

Sección 10 — Plantillas de Boletines Escolares

src/templates/boletines/types.ts

Interfaz de datos para la generación de boletines. Estructura las calificaciones por periodo (P1–P4), promedio de competencia (PC), y calificación final por asignatura para cada estudiante.

```
export interface BoletinTemplateProps {
  curso: any;
  estudiantes: any[];
  docenteNombre: string;
  studentGrades: Record<number, Record<string, {
    P1: Record<BCKey, number | null>;
    P2: Record<BCKey, number | null>;
    P3: Record<BCKey, number | null>;
    P4: Record<BCKey, number | null>;
    PC: Record<BCKey, number | null>;
    finalGrade: number | null;
  }>;
}
state: AppState;
```

src/templates/boletines/Boletin2do.tsx (fragmento: estructura del boletín oficial)

Plantilla de Boletín de Calificaciones de 2do Grado del Nivel Primario, con diseño de impresión landscape (11×8.5in) conforme al formato del MINERD (Ministerio de Educación de la República Dominicana). Incluye portada con datos del centro, tabla de competencias por asignatura/periodo, y sección de observaciones.

```
export default function Boletin2do({ curso, estudiantes, docenteNombre,
  studentGrades, state }: BoletinTemplateProps) {
  return (
    <>
      <style dangerouslySetInnerHTML={{ __html: `
        @page { size: 11in 8.5in; margin: 0.3in; }
        .page-landscape { width: 11in; min-height: 8.5in; }
        :root { --navy: #1c3f5f; --lightblue: #cfe3f2; }
        .header-bar { background: var(--navy); color: #fff; }
        .boletin-title { color: #0072bc; font-weight: bold; }
        @media print { .page { page-break-after: always; } }
      `}} />

      {estudiantes.map((est) => (
        <div key={est.id}>
          {/* Página 1: Portada con datos del centro educativo */}
          <div className="page landscape">
            <div>REPÚBLICA DOMINICANA</div>
            <div>MINISTERIO DE EDUCACIÓN</div>
            <div>INFORME DE APRENDIZAJE</div>
            {/* Campos: Centro, Regional, Distrito, Grado, etc. */}
          </div>

          {/* Página 2: Tabla de competencias fundamentales */}
          <div className="page landscape">
            {/* Filas: Asignaturas × Columnas: BC1-BC4 × Periodos P1-P4 + PC */}
          </div>
        </div>
      ))}
    </>
  );
}
```

Sección 11 — Backend: API de Autenticación

server/server.js

Servidor Express.js con 3 endpoints de autenticación segura: solicitud de recuperación de contraseña (con anti-abuso, timing attack protection, tokens criptográficos de un solo uso con expiración de 30 minutos), validación de tokens, y restablecimiento de contraseña con validación de complejidad. Incluye auditoría de eventos de seguridad, limitación de tasa (rate limiting) y configuración SMTP para envío de correos.

```
import express from 'express';
import cors from 'cors';
import helmet from 'helmet';
import crypto from 'crypto';
import nodemailer from 'nodemailer';
import { createClient } from '@supabase/supabase-js';
import { rateLimit } from 'express-rate-limit';

const supabaseUrl = '*****';
const supabaseServiceKey = '*****';

const supabaseAdmin = createClient(supabaseUrl, supabaseServiceKey, {
  auth: { autoRefreshToken: false, persistSession: false }
});

// Helper para registrar auditoría de eventos de seguridad
async function registrarAuditoria(userId, email, ip, evento, detalles) {
  await supabaseAdmin.from('auditoria_recuperacion').insert({
    user_id: userId, email, ip_address: ip,
    evento, detalles: detalles || {}
  });
}

/**
 * Endpoint: Solicitar recuperación de contraseña
 * Seguridad: rate limiting, detección de fuerza bruta, timing attack protection
 */
app.post('/api/auth/forgot-password', forgotPasswordLimiter, async (req, res) => {
  const { email } = req.body;
  const ip = req.ip;

  // Control de abuso: máximo 3 envíos por correo cada 15 minutos
  const { data: enviosRecientes } = await supabaseAdmin
    .from('auditoria_recuperacion').select('id')
    .eq('email', cleanEmail).eq('evento', 'solicitud_enviada')
    .gt('created_at', new Date(Date.now() - 15 * 60 * 1000).toISOString());

  if (enviosRecientes?.length >= 3) {
    await registrarAuditoria(null, cleanEmail, ip, 'fuerza_bruta_detectada');
    return res.status(200).json({ message: responseMsg });
  }

  // Generar token criptográficamente seguro (32 bytes, hex)
  const token = crypto.randomBytes(32).toString('hex');
  const expiresAt = new Date(Date.now() + 30 * 60 * 1000); // 30 minutos

  await supabaseAdmin.from('recuperacion_tokens').insert({
    user_id: userId, token, expires_at: expiresAt.toISOString(), estado: 'activo'
  });
});

/**
 * Endpoint: Restablecer contraseña con token válido
 * Validación: complejidad de contraseña, estado del token, expiración
 */
app.post('/api/auth/reset-password', resetPasswordLimiter, async (req, res) => {
  const passwordRegex = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d){8}$/;
  if (!passwordRegex.test(password)) {
    return res.status(400).json({ error: 'La contraseña no cumple requisitos de seguridad' });
  }

  // Cambiar contraseña via API administrativa de Supabase

```

```

await supabaseAdmin.auth.admin.updateUserById(userId, { password });

// Marcar token como usado e invalidar tokens remanentes
await supabaseAdmin.from('recuperacion_tokens')
  .update({ estado: 'usado' }).eq('id', record.id);
await supabaseAdmin.rpc('revocar_tokens_activos', { p_user_id: userId });
});

app.listen(PORT, () => console.log(`🚀 Servidor corriendo en puerto ${PORT}`));

```

Sección 12 — Esquema de Base de Datos y Seguridad SQL

supabase_fix.sql (fragmento: políticas RLS y función de seguridad)

Esquema de políticas de seguridad a nivel de fila (Row Level Security) que garantiza aislamiento de datos por usuario. Incluye la función personalizada `is_course_teacher()` que verifica pertenencia a un curso como propietario o co-docente.

```

-- Función auxiliar para configurar RLS estándar en cada tabla
CREATE OR REPLACE FUNCTION setup_user_rols(table_name TEXT) RETURNS VOID AS $$
BEGIN
  EXECUTE format('ALTER TABLE public.%I ENABLE ROW LEVEL SECURITY', table_name);
  EXECUTE format('CREATE POLICY %I_select ON public.%I FOR SELECT
    TO authenticated USING (auth.uid() = user_id)', table_name, table_name);
  EXECUTE format('CREATE POLICY %I_insert ON public.%I FOR INSERT
    TO authenticated WITH CHECK (auth.uid() = user_id)', table_name, table_name);
  EXECUTE format('CREATE POLICY %I_update ON public.%I FOR UPDATE
    TO authenticated USING (auth.uid() = user_id)
    WITH CHECK (auth.uid() = user_id)', table_name, table_name);
END;
$$ LANGUAGE plpgsql;

-- Aplicar RLS a todas las tablas del sistema educativo
SELECT setup_user_rols('cursos');
SELECT setup_user_rols('estudiantes');
SELECT setup_user_rols('actividades');
SELECT setup_user_rols('calificaciones');
SELECT setup_user_rols('recuperaciones');
SELECT setup_user_rols('evaluaciones_rubrica');
SELECT setup_user_rols('evaluaciones_cotejo');

-- Función de verificación de pertenencia a curso (SECURITY DEFINER)
CREATE OR REPLACE FUNCTION public.is_course_teacher(p_curso_id bigint)
RETURNS boolean AS $$
DECLARE
  current_user_id uuid;
BEGIN
  current_user_id := auth.uid();
  RETURN EXISTS (
    SELECT 1 FROM public.cursos
    WHERE id = p_curso_id AND user_id = current_user_id
  ) OR EXISTS (
    SELECT 1 FROM public.curso_docentes
    WHERE curso_id = p_curso_id AND docente_id = current_user_id
  );
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

-- Tabla de tokens de recuperación de contraseña
CREATE TABLE IF NOT EXISTS public.recuperacion_tokens (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE NOT NULL,
  token TEXT UNIQUE NOT NULL,
  created_at TIMESTAMPTZ DEFAULT now() NOT NULL,
  expires_at TIMESTAMPTZ NOT NULL,
  estado TEXT DEFAULT 'activo' NOT NULL
  CHECK (estado IN ('activo', 'usado', 'revocado'))
);

-- Función para revocar tokens activos de un usuario (prevención de fugas)

```

```
CREATE OR REPLACE FUNCTION public.revocar_tokens_activos(p_user_id uuid)
RETURNS void AS $$
BEGIN
    UPDATE public.recuperacion_tokens
    SET estado = 'revocado'
    WHERE user_id = p_user_id AND estado = 'activo';
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;
```

Portfolio Pro (Noether SaaS) — Sistema de Evaluación por Competencias

Soporte Material de la Obra para registro ante ONDA — República Dominicana

Documento generado el 30 de julio de 2026. Todos los derechos reservados.

Este documento contiene fragmentos representativos del código fuente original del autor.